B.Sc. (General/Special) Degree Programme
Faculty of Applied Sciences
University of Sri Jayewardenepura

ICT 1022 2.0 Computer System Architecture

Presented By:
Surani Tissera(PhD)
Department of Computer Science

INSTRUCTION FORMAT AND ADDRESSING MODES

Instruction Format

An instruction consists of two parts:-

a) Operation Code or Opcode

- Specifies the operation to be performed by the instruction.
- Various categories of instructions: data transfer, arithmetic and logical, control, I/O and special machine control.

b) Operand(s)

- Specifies the source(s) and destination of the operation.
- Source operand can be specified by an immediate data, by naming a register, or specifying the address of memory.
- Destination can be specified by a register or memory address

Instruction Format

- Number of operands varies from instruction to instruction.
- Also for specifying an operand, various addressing modes are possible:
  - Immediate addressing
  - Direct addressing
  - Indirect addressing
  - Relative addressing
  - Indexed addressing, and many more.

Instruction Format Examples

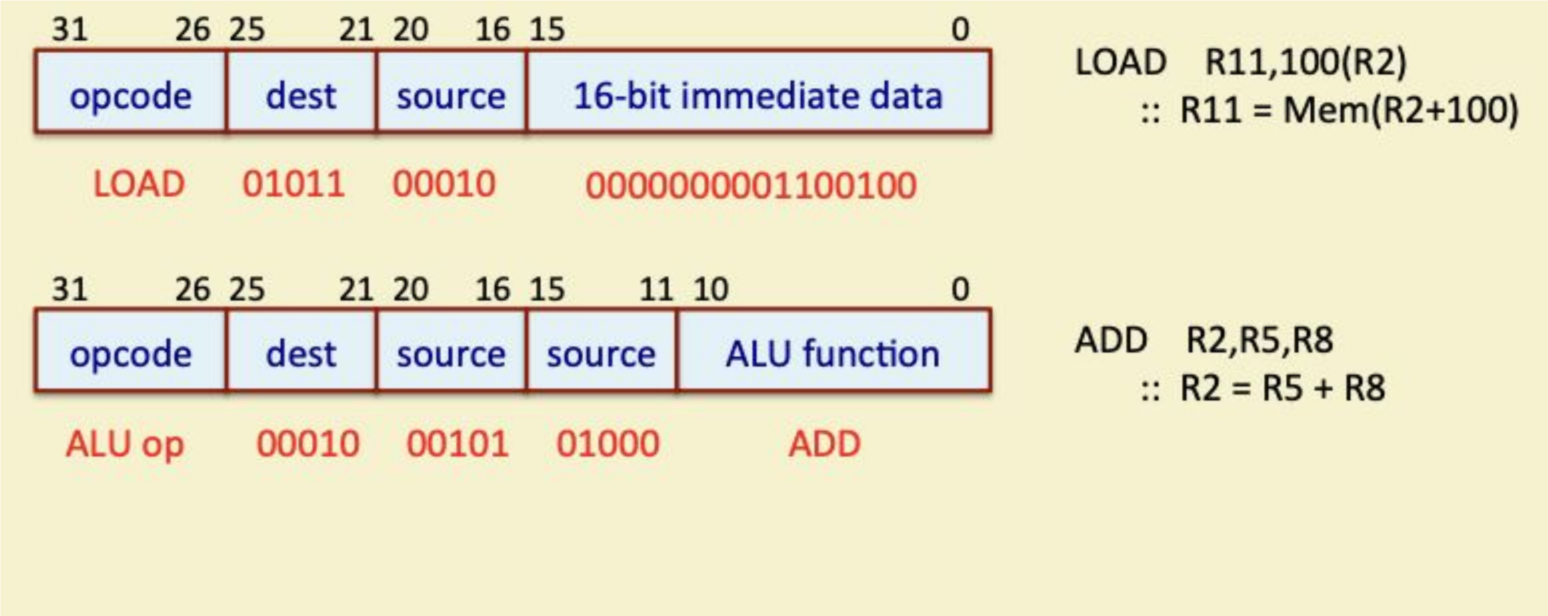
opcode |	Implied addressing: NOP, HALT
opcode | memory address	1-address: ADD X, LOAD M
opcode | memory address | memory address	2-address: ADD X,Y
opcode | register | memory address	Register-memory: ADD R1,X
opcode | register | register | register	Register-register: ADD R1,R2,R3

A 32-bit Instruction Example

- Suppose we have an ISA with 32-bit instructions only.
  - Fixed size instructions make the decoding easier.

- Some instruction encoding examples are shown.
  - Assume that there are 32 registers R0 to R31, all of 32-bits.
  - 5-bits are required to specify a register.

A 32-bit Instruction Example



ADDRESSING MODES

What are Addressing Modes?

- They specify the mechanism by which the operand data can be located.
- Some ISA's are quite complex and supports many addressing modes.
- ISA's based on load-store architecture are usually simple and support very
- limited number of addressing modes.
- Various addressing modes exist:

  – Immediate, Direct, Indirect, Register, Register Indirect, Indexed, Stack, Relative, Autoincrement, Autodecrement, Based, etc.

  – Not all processors support all addressing modes.

  – Wesh all briefly look at the common addressing modes and how they work.

Immediate Addressing

The operand is part of the instruction itself.

– No memory reference is required to access the operand.

– Fast but limited range (because a limited number of bits are provided to specify the immediate data).

- Examples:

– ADDI #25 //ACC=ACC+25

– ADDI R1,R2,42 // R1 = R2 + 42

Direct Addressing

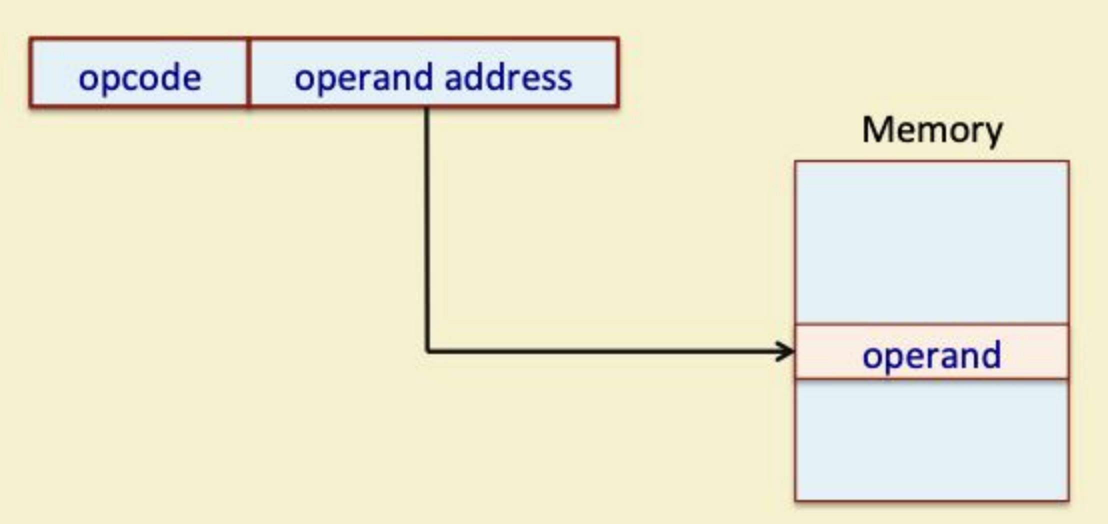
- The instruction contains a field that holds the memory address of the operand.
- Examples:

– ADD R1,20A6H // R1 = R1 + Mem[20A6]

- Single memory access is required to access the operand.

– No additional calculations required to determine the operand address.

– Limited address space (as number of bits is limited, say, 16 bits).

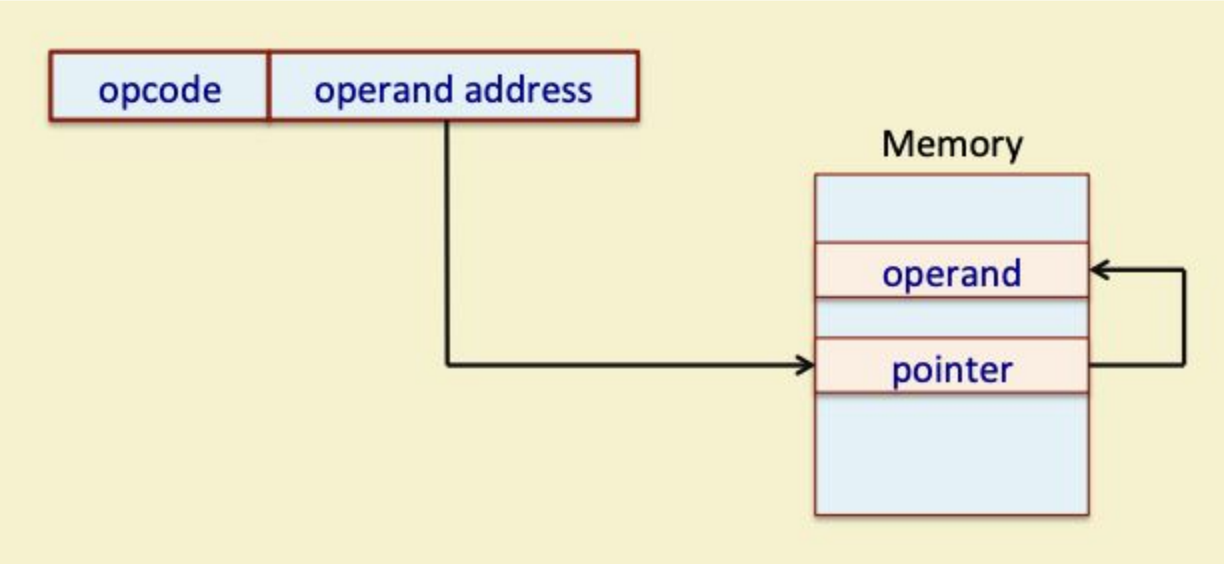

opcode
operand address
Memory
operand

Indirect Addressing

- The instruction contains a field that holds the memory address, which in turn holds the memory address of the operand.
- Two memory accesses are required to get the operand value.
- Slower but can access large address space.
  – Not limited by the number of bits in operand address like direct addressing.

- Examples:
  – ADD R1,(20A6H) // R1 = R1 + (Mem[20A6])

Indirect Addressing



Register Addressing

- The operand is held in a register, and the instruction specifies the register number.

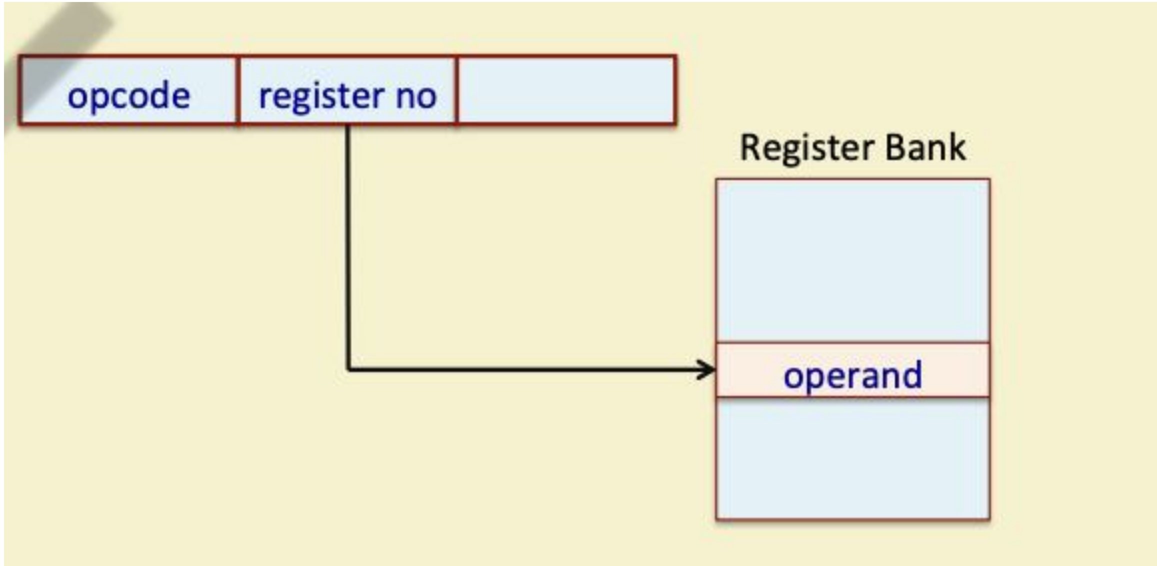
  – Very few number of bits needed, as the number of registers is limited

  – Faster execution,since no memory access is required for getting the operand.

- Modern load-store architectures support large number of registers.
- Examples:

  – ADD R1,R2,R3 //R1=R2+R3

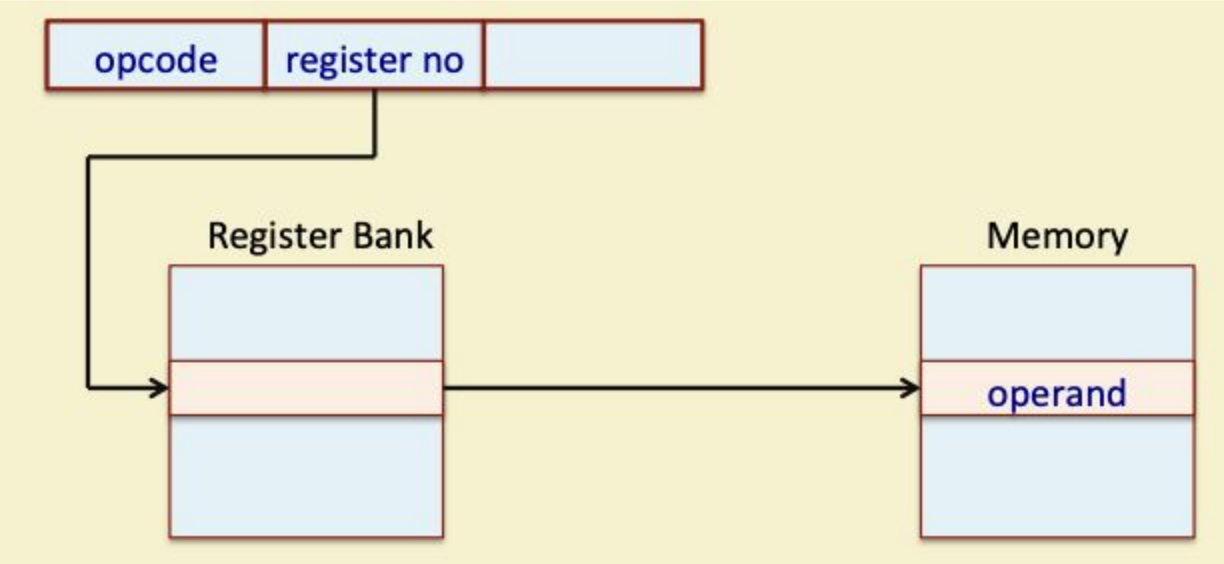
  – MOV R2,R5 // R2 = R5


opcode
register no
Register Bank
operand

Register Indirect Addressing

- The instruction specifies a register, and the register holds the memory address where the operand is stored.
  - Can access large address space.
  - One fewer memory access as compared to indirect addressing.
- Example:
  - ADD R1,(R5) // PC = R1 + Mem[R5]

Register Indirect Addressing

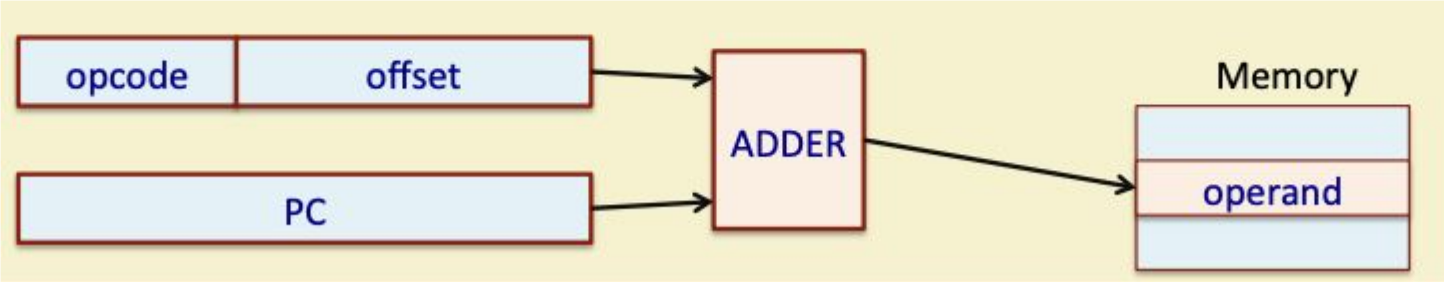


Relative Addressing (PC Relative)

The instruction specifies an offset of displacement, which is added to the program counter (PC) to get the effective address of the operand.

– Since the number of bits to specify the offset is limited, the range of relative addressing is also limited.

– If a12-bit off set is specified, it can have values ranging from -2048 to +2047.



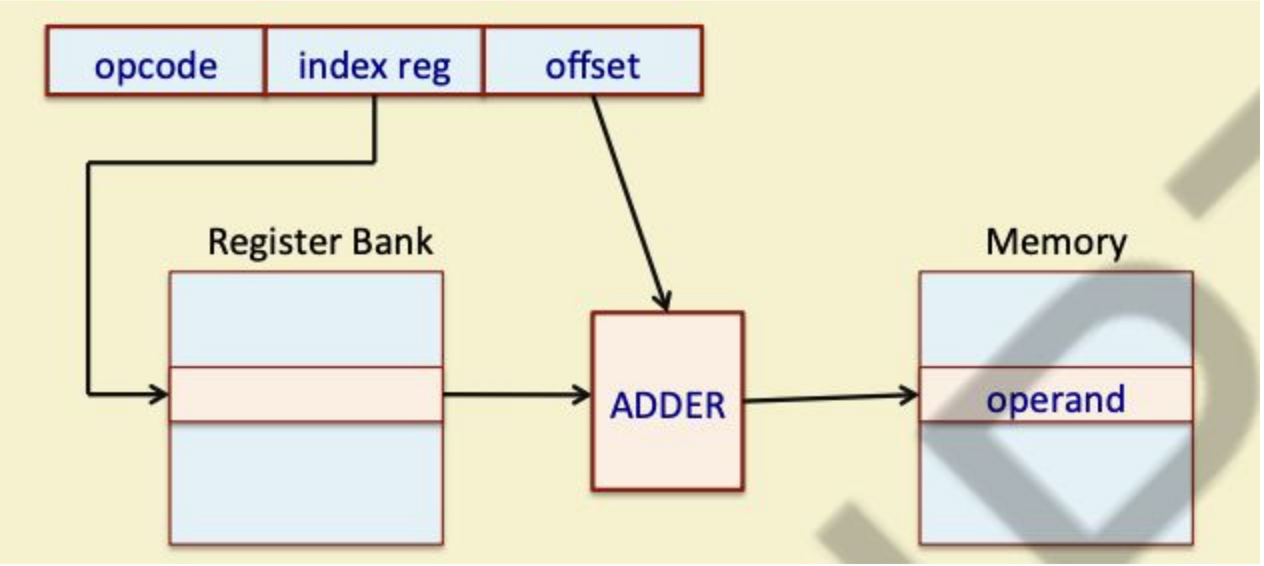
Indexed Addressing

- Either a special-purpose register, or a general-purpose register, is used as index register in this addressing mode.
- The instruction specifies an offset of displacement, which is added to the index register to get the effective address of the operand.

- Example:
  - LOAD R1,1050(R3) // R1 = Mem[1050+R3]

- Can be used to sequentially access the elements of an array.
  - Offset gives the starting address of the array, and the index register value specifies the array element to be used.

Indexed Addressing



Stack Addressing

- Operand is implicitly on top of the stack.
- Used in zero-address machines earlier.
- Examples:
  - ADD
  - PUSH X
  - POP X
- Many processors have a special register called the stack pointer (SP) that keeps track of the stack-top in memory.

– PUSH,POP,CALL,RET instructions automatically modify SP.

Some Other Addressing Modes

- Base addressing
  – The processor has a special register called the base register or segment register.
  – All operand addresses generated are added to the base register to get the final
  memory address.
  – Allows easy movement of code and data in memory.

- Autoincrement and Autodecrement
  – First introduced in the PDP-11 computer system.
  – The register holding the operand address is automatically incremented or decremented after accessing the operand (like a++ and a-- in C).

CISC AND RISC ARCHITECTURE

Broad Classification

- Computer architectures have evolved over the years.
  - Features that were developed for mainframes and supercomputers in the 1960s and 1970s have started to appear on a regular basis on later generation microprocessors.

- Two broad classifications of ISA:
  a) Complex Instruction Set Computer (CISC)
  b) Reduced Instruction Set Computer (RISC)

CISC versus RISC Architectures

Complex Instruction Set Computer (CISC)

- More traditional approach.
- Main features:
  - Complex instruction set
  - Large number of addressing modes (R-R, R-M, M-M, indexed, indirect, etc.)
  - Special-purpose registers and Flags (sign, zero, carry, overflow, etc.)
  - Variable-length instructions / Complex instruction encoding
  - Ease of mapping high-level language statements to machine instructions
  - Instruction decoding / control unit design more complex
  - Pipeline implementation quite complex

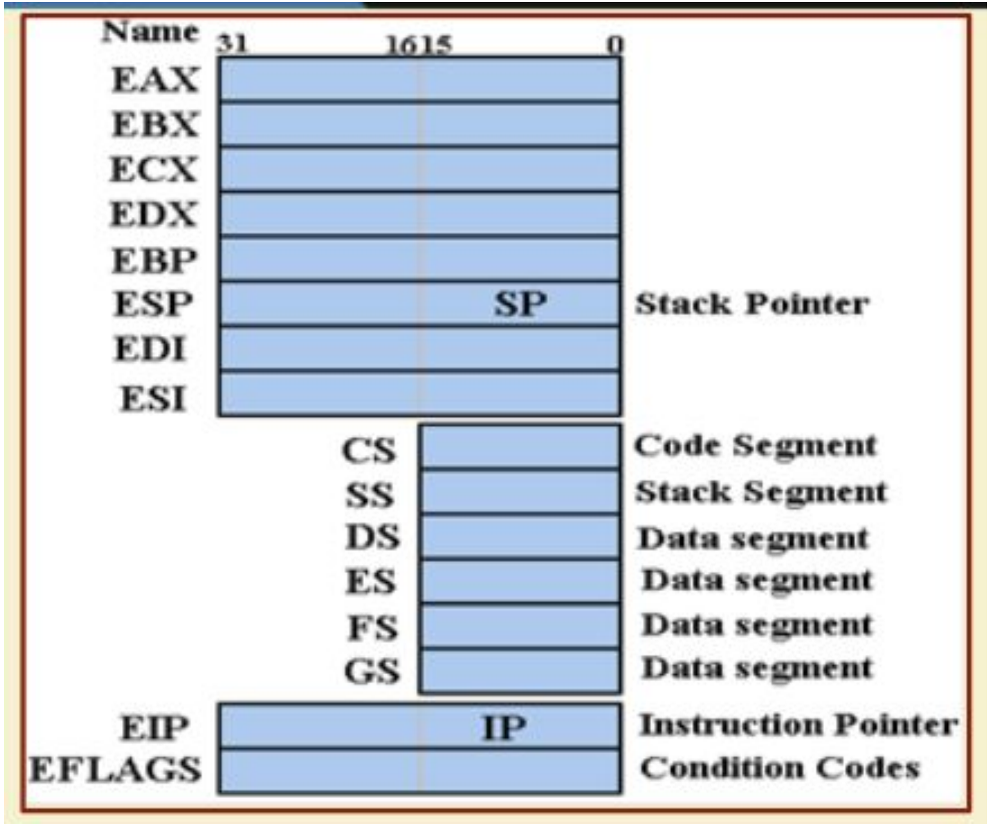
CISC versus RISC Architectures

– CISC Examples:
  - IBM 360/370 (1960-70)
  - VAX-11/780 (1970-80)
  - Intel x86 / Pentium (1985-present)

Only CISC instruction set that survived over generations.

- Desktop PC's / Laptops use these.
- The volume of chips manufactured is so high that there is enough motivation to pay the extra design cost.
- Sufficient hardware resources available today to translate from CISC to RISC internally.

Register Set in Pentium



Addressing Modes in VAX

Addressing Mode	Example	Micro-operation
Register direct	ADD R1,R2	R1 = R1 + R2
Immediate	ADD R1,#15	R1 = R1 + 15
Displacement	ADD R1,220(R5)	R1 = R1 + Mem[220+R5]
Register indirect	ADD R1,(R3)	R1 = R1 + Mem[R3]
Indexed	ADD R1,(R2+R3)	R1 = R1 + Mem[R2+R3]
Direct	ADD R1, (1000)	R1 = R1 + Mem[1000]
Memory indirect	ADD R1,@(R4)	R1 = R1 + Mem[Mem[R4]]
Autoincrement	ADD R1,(R2)+	R1 = R1 + Mem[R2]; R2++
Autodecrement	ADD R1,(R2)-	R1 = R1 + Mem[R2]; R2--
Scaled	ADD R1,50(R2)[R3]	R1 = R1 + Mem[50+R2+R3*d]

Addressing Modes in VAX

Addressing Mode	Example	Micro-operation
Register direct	ADD R1,R2	R1 = R1 + R2
Immediate	ADD R1,#15	R1 = R1 + 15
Displacement	ADD R1,220(R5)	R1 = R1 + Mem[220+R5]
Register indirect	ADD R1,(R3)	R1 = R1 + Mem[R3]
Indexed	ADD R1,(R2+R3)	R1 = R1 + Mem[R2+R3]
Direct	ADD R1, (1000)	R1 = R1 + Mem[1000]
Memory indirect	ADD R1,@(R4)	R1 = R1 + Mem[Mem[R4]]
Autoincrement	ADD R1,(R2)+	R1 = R1 + Mem[R2]; R2++
Autodecrement	ADD R1,(R2)-	R1 = R1 + Mem[R2]; R2--
Scaled	ADD R1,50(R2)[R3]	R1 = R1 + Mem[50+R2+R3*d]

Reduced Instruction Set Computer (RISC)

- Very widely used among many manufacturers today.
- Also referred to as Load-Store Architecture.
  - Only LOAD and STORE instructions access memory.
  - All other instructions operate on processor registers.
- Main features:
  - Simple architecture for the sake of efficient pipelining.
  - Simple instruction set with very few addressing modes.
  - Large number of general-purpose registers; very few special-purpose.
  - Instruction length and encoding uniform for easy instruction decoding.
  - Compiler assisted scheduling of pipeline for improved performance.

– RISC Examples:

- CDC 6600 (1964)
- MIPS family (1980-90)
- SPARC
- ARM microcontroller family

- Almost all the computers today use a RISC based pipeline for efficient implementation.
    - RISC based computers use compilers to translate into RISC instructions.
    - CISC based computers (e.g. x86) use hardware to translate into RISC instructions.

Results of a Comparative Study

- A quantitative comparison of VAX 8700 (a CISC machine) and MIPS M2000 (a RISC machine) with comparable organizations was carried out in 1991.
- Some findings:
  - MIPS required execution of about twice the number of instructions as compared to VAX.
  - Cycles Per Instructions(CPI) for VAX was about six times larger than that of MIPS.
  - Hence, MIPS had three times the performance of VAX.
  - Also,much less hardware is required to build MIPS as compared to VAX.

Conclusion:

– Persisting with CISC architecture is too costly, both in terms of hardware cost and also performance.

– VAX was replaced by ALPHA (a RISC processor) by Digital Equipment Corporation (DEC).

– CISC architecture based on x86 is different.

Because of huge number of installed base, backward compatibility of machine code is very important from commercial point of view.

They have adopted a balanced view: (a) user's view is a CISC instruction set, (b) hardware translates every CISC instruction into an equivalent set of RISC instructions internally, (c) an instruction pipeline executes the RISC instructions efficiently.

MIPS32 Architecture: A Case Study

As a case study of RISC ISA, we shall be considering the MIPS32 architecture.

- Look into the instruction set and instruction encoding in detail.
- Design the data path of the MIPS32 architecture, and also look into the control unit design issues.
- Extend the basic data path of MIPS32 to a pipeline architecture, and discuss some of the issues therein.

MIPS32 CPU Registers

The MIPS32 ISA defines the following CPU registers that are visible to the machine/assembly language programmer.

a) 32,32-bit general purpose registers(GPRs),R0 to R31.

b) A special-purpose 32-bit program counter (PC).

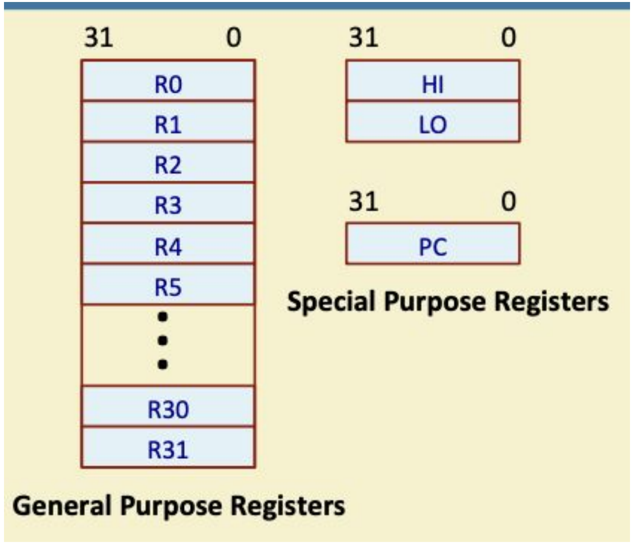
- Points to the next instruction in memory to be fetched and executed.
- Not directly visible to the programmer.
- Affected only indirectly by certain instructions (like branch, call, etc.)

c) A pair of 32-bit special-purpose registers HI and LO, which are used to hold the results of multiply, divide, and multiply-accumulate instructions.

MIPS32 CPU Registers

- Some common registers are missing in MIPS32.
  - Stack Pointer (SP) register, which helps in maintaining a stack in main memory.
    - Any of the GPRs can be used as the stack pointer.
    - No separate PUSH, POP, CALL and RET instructions.
  - Index Register (IX), which helps in accessing memory words sequentially in memory.
    - Any of the GPRs can be used as an index register.
  - Flag registers (like ZERO, SIGN, CARRY, OVERFLOW) that keeps track of the results of arithmetic and logical operations.
    - Maintains flags in registers, to avoid problems in pipeline implementation.

MIPS32 CPU Registers



Two of the GPRs have assigned functions:

a) R0 is hard-wired to a value of zero.
   - Can be used as the target register for any instruction whose result is to be discarded.
   - Can also be used as a source when a zero value is needed.

b) R31 is used to store the return address when a function call is made.
   - Used by the jump-and-link and branch-and-link instructions like JAL, BLTZAL, BGEZAL, etc.
   - Can also be used as a normal register.

Some Examples

```
LD R4, 50(R3)        // R4 = Mem[50+R3]
ADD R2,R1,R4         //R2=R1+R4
SD 54(R3), R2        // Mem[54+R3] = R2
```

```
ADD R2,R5,R0 //R2=R5
```

```
MAIN:  ADDI   R1, R0, 35 // R1 = 35
       ADDI   R2, R0, 56 // R2 = 56

       JAL    GCD .....
GCD:   .....  //Find GCD of R1 & R2
       JR     R31
```

Some Examples

How are the HI and LO registers used?

- During a multiply operation, the HI and LO registers store the product of an integer multiply.
  - HI denotes the high-order 32 bits, and LO denotes the low-order 32 bits.
- During a multiply-add or multiply-subtract operation, the HI and LO registers store the result of the integer multiply-add or multiply-subtract.
- During a division, the HI and LO registers store the quotient (in LO) and remainder (in HI) of integer divide.

Some MIPS32 Assembly Language Conventions

- The integer registers of MIPS32 can be accessed as R0..R31 or r0..r31 in an assembly language program.
- Several assemblers and simulators are available in the public domain (like QtSPIM) that follow some specific conventions.
  – These conventions have become like a de facto standard when we write assembly language programs for MIPS32.
  – Basically some alternate names are used for the registers to indicate their intended usage.

Register name	Register number	Usage
$zero	R0	Constant zero

Used to represent the constant zero value, wherever required in a program.

Register name	Register number	Usage
$at	R1	Reserved for assembler

May be used as temporary register during macro expansion by assembler.

- Assembler provides an extension to the MIPS32 instruction set that are converted to standard MIPS32 instructions.

Example: Load Address instruction used to initialize pointers

```
la      R5,addr
   ⇩
lui     $at,Upper-16-bits-of-addr
ori     R5,$at,Lower-16-bits-of-addr
```

Register name	Register number	Usage
$v0	R2	Result of function, or for expression evaluation
$v1	R3	Result of function, or for expression evaluation

May be used for up to two function return values, and also as temporary registers during expression evaluation.

Register name	Register number	Usage
$a0	R4	Argument 1
$a1	R5	Argument 2
$a2	R6	Argument 3
$a3	R7	Argument 3

May be used to pass up to four arguments to functions.

Register name	Register number	Usage
$t0	R8	Temporary (not preserved across call)
$t1	R9	Temporary (not preserved across call)
$t2	R10	Temporary (not preserved across call)
$t3	R11	Temporary (not preserved across call)
$t4	R12	
$t5	R13	
$t6	R14	
$t7	R15	
$t8	R24	Temporary (not preserved across call)
$t9	R25	Temporary (not preserved across call)

May be used as temporary variables in programs.

These registers might get modified when some functions are called (other than user-written functions).

Register name	Register number	Usage
$s0	R16	Temporary (preserved across call)
$s1	R17	Temporary (preserved across call)
$s2	R18	Temporary (preserved across call)
$s3	R19	Temporary (preserved across call)
$s4	R20	Temporary (preserved across call)
$s5	R21	Temporary (preserved across call)
$s6	R22	
$s7	R23	

May be used as temporary variables in programs.

These registers do not get modified across function calls.

Register name	Register number	Usage
$gp	R28	Pointer to global area
$sp	R29	Stack pointer
$fp	R30	Frame pointer
$ra	R31	Return address (used by function call)

These registers are used for a variety of pointers:

- Global area: points to the memory address from where the global variables are allocated space.
- Stack pointer: points to the top of the stack in memory.
- Frame pointer: points to the activation record in stack.
- Return address: used while returning from a funcFon.

Register name	Register number	Usage
$k0	R26	Reserved for OS kernel
$k1	R27	Reserved for OS kernel

- These registers are supposed to be used by the OS kernel in a real computer system.
- It is highly recommended not to use these registers.


Thank You!